NOUS AI ACADEMY / CORE LIBRARY

Algorithms, Data Structures, and AI Problem Solving

Efficient reasoning from arrays to graph search

Nous AI Academy

TEXTBOOK EDITION 2 / 150 PAGES / OFFLINE

Original curriculum - technical and editorial review recommended

BOOK-03

FRONT MATTER

How to Use This Book

EDITORIAL GUIDANCE

Read actively. Predict before revealing a worked step, reproduce examples locally, keep a mistake notebook, and complete mastery checks without notes before moving forward.

READER NOTE

Use this book as a sequence, annotate key arguments, reproduce the examples, and keep a separate notebook for attempts, corrections, and reflections.

FRONT MATTER

Prerequisites and Outcomes

EDITORIAL GUIDANCE

Prerequisites: Python foundations or equivalent programming experience.

Outcomes: Choose, implement, analyze, test, and explain efficient algorithms and data structures.

READER NOTE

Use this book as a sequence, annotate key arguments, reproduce the examples, and keep a separate notebook for attempts, corrections, and reflections.

FRONT MATTER

Learning Path

EDITORIAL GUIDANCE

Each chapter contains ten pages: orientation, first-principles model, language and notation, two worked examples, implementation lab, failure modes, guided practice, independent practice, and mastery check.

READER NOTE

Use this book as a sequence, annotate key arguments, reproduce the examples, and keep a separate notebook for attempts, corrections, and reflections.

CHAPTER 01 / ORIENTATION AND QUESTIONS

Complexity and Measurement: Orientation and Questions

Explain and apply complexity and measurement using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Complexity describes how time and memory grow with input size, allowing implementations to be compared beyond one machine. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Complexity, Measurement are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should complexity and measurement be used, and what observable evidence would make that choice defensible? Count operations for a single loop, a nested loop, and binary search.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Complexity and Measurement in one precise sentence and name one assumption.
2. Create a two-step example of complexity and measurement and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 01 / FIRST-PRINCIPLES MODEL

Complexity and Measurement: First-Principles Model

Explain and apply complexity and measurement using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Complexity describes how time and memory grow with input size, allowing implementations to be compared beyond one machine. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to complexity and measurement.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Count operations for a single loop, a nested loop, and binary search.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Complexity and Measurement in one precise sentence and name one assumption.
2. Create a two-step example of complexity and measurement and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 01 / LANGUAGE AND NOTATION

Complexity and Measurement: Language and Notation

Explain and apply complexity and measurement using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for complexity and measurement should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for complexity and measurement.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Complexity and Measurement in one precise sentence and name one assumption.
2. Create a two-step example of complexity and measurement and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 01 / WORKED EXAMPLE A

Complexity and Measurement: Worked Example A

Explain and apply complexity and measurement using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Count operations for a single loop, a nested loop, and binary search. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns complexity and measurement into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Complexity and Measurement in one precise sentence and name one assumption.
2. Create a two-step example of complexity and measurement and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 01 / WORKED EXAMPLE B

Complexity and Measurement: Worked Example B

Explain and apply complexity and measurement using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Count operations for a single loop, a nested loop, and binary search. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns complexity and measurement into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Complexity and Measurement in one precise sentence and name one assumption.
2. Create a two-step example of complexity and measurement and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 01 / IMPLEMENTATION LAB

Complexity and Measurement: Implementation Lab

Explain and apply complexity and measurement using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of complexity and measurement into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Count operations for a single loop, a nested loop, and binary search. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Complexity and Measurement in one precise sentence and name one assumption.
2. Create a two-step example of complexity and measurement and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 01 / FAILURE MODES

Complexity and Measurement: Failure Modes

Explain and apply complexity and measurement using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how complexity and measurement can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to complexity and measurement. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Complexity and Measurement in one precise sentence and name one assumption.
2. Create a two-step example of complexity and measurement and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 01 / GUIDED PRACTICE

Complexity and Measurement: Guided Practice

Explain and apply complexity and measurement using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for complexity and measurement removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Count operations for a single loop, a nested loop, and binary search.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Complexity and Measurement in one precise sentence and name one assumption.
2. Create a two-step example of complexity and measurement and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 01 / INDEPENDENT PRACTICE

Complexity and Measurement: Independent Practice

Explain and apply complexity and measurement using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new complexity and measurement task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Count operations for a single loop, a nested loop, and binary search. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Complexity and Measurement in one precise sentence and name one assumption.
2. Create a two-step example of complexity and measurement and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 01 / MASTERY CHECK

Complexity and Measurement: Mastery Check

Explain and apply complexity and measurement using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of complexity and measurement means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain complexity describes how time and memory grow with input size, allowing implementations to be compared beyond one machine. Then solve and verify this scenario: Count operations for a single loop, a nested loop, and binary search.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Complexity and Measurement in one precise sentence and name one assumption.
2. Create a two-step example of complexity and measurement and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 02 / ORIENTATION AND QUESTIONS

Arrays, Strings, and Two Pointers: Orientation and Questions

Explain and apply arrays, strings, and two pointers using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Contiguous sequences support fast indexed access, while pointer patterns reduce repeated scanning. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Arrays, Strings, Pointers are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should arrays, strings, and two pointers be used, and what observable evidence would make that choice defensible? Find a target pair in a sorted array using two moving indices.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Arrays, Strings, and Two Pointers in one precise sentence and name one assumption.
2. Create a two-step example of arrays, strings, and two pointers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 02 / FIRST-PRINCIPLES MODEL

Arrays, Strings, and Two Pointers: First-Principles Model

Explain and apply arrays, strings, and two pointers using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Contiguous sequences support fast indexed access, while pointer patterns reduce repeated scanning. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to arrays, strings, and two pointers.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Find a target pair in a sorted array using two moving indices.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Arrays, Strings, and Two Pointers in one precise sentence and name one assumption.
2. Create a two-step example of arrays, strings, and two pointers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 02 / LANGUAGE AND NOTATION

Arrays, Strings, and Two Pointers: Language and Notation

Explain and apply arrays, strings, and two pointers using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for arrays, strings, and two pointers should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for arrays, strings, and two pointers.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Arrays, Strings, and Two Pointers in one precise sentence and name one assumption.
2. Create a two-step example of arrays, strings, and two pointers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 02 / WORKED EXAMPLE A

Arrays, Strings, and Two Pointers: Worked Example A

Explain and apply arrays, strings, and two pointers using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Find a target pair in a sorted array using two moving indices. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns arrays, strings, and two pointers into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Arrays, Strings, and Two Pointers in one precise sentence and name one assumption.
2. Create a two-step example of arrays, strings, and two pointers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 02 / WORKED EXAMPLE B

Arrays, Strings, and Two Pointers: Worked Example B

Explain and apply arrays, strings, and two pointers using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Find a target pair in a sorted array using two moving indices. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns arrays, strings, and two pointers into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Arrays, Strings, and Two Pointers in one precise sentence and name one assumption.
2. Create a two-step example of arrays, strings, and two pointers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 02 / IMPLEMENTATION LAB

Arrays, Strings, and Two Pointers: Implementation Lab

Explain and apply arrays, strings, and two pointers using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of arrays, strings, and two pointers into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Find a target pair in a sorted array using two moving indices. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Arrays, Strings, and Two Pointers in one precise sentence and name one assumption.
2. Create a two-step example of arrays, strings, and two pointers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 02 / FAILURE MODES

Arrays, Strings, and Two Pointers: Failure Modes

Explain and apply arrays, strings, and two pointers using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how arrays, strings, and two pointers can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to arrays, strings, and two pointers. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Arrays, Strings, and Two Pointers in one precise sentence and name one assumption.
2. Create a two-step example of arrays, strings, and two pointers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 02 / GUIDED PRACTICE

Arrays, Strings, and Two Pointers: Guided Practice

Explain and apply arrays, strings, and two pointers using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for arrays, strings, and two pointers removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Find a target pair in a sorted array using two moving indices.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Arrays, Strings, and Two Pointers in one precise sentence and name one assumption.
2. Create a two-step example of arrays, strings, and two pointers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 02 / INDEPENDENT PRACTICE

Arrays, Strings, and Two Pointers: Independent Practice

Explain and apply arrays, strings, and two pointers using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new arrays, strings, and two pointers task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Find a target pair in a sorted array using two moving indices. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Arrays, Strings, and Two Pointers in one precise sentence and name one assumption.
2. Create a two-step example of arrays, strings, and two pointers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 02 / MASTERY CHECK

Arrays, Strings, and Two Pointers: Mastery Check

Explain and apply arrays, strings, and two pointers using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of arrays, strings, and two pointers means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain contiguous sequences support fast indexed access, while pointer patterns reduce repeated scanning. Then solve and verify this scenario: Find a target pair in a sorted array using two moving indices.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Arrays, Strings, and Two Pointers in one precise sentence and name one assumption.
2. Create a two-step example of arrays, strings, and two pointers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 03 / ORIENTATION AND QUESTIONS

Stacks and Queues: Orientation and Questions

Explain and apply stacks and queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Stacks model last-in-first-out work; queues model first-in-first-out work and staged processing. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Stacks, Queues are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should stacks and queues be used, and what observable evidence would make that choice defensible? Validate nested brackets with a stack and traverse states with a queue.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Stacks and Queues in one precise sentence and name one assumption.
2. Create a two-step example of stacks and queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 03 / FIRST-PRINCIPLES MODEL

Stacks and Queues: First-Principles Model

Explain and apply stacks and queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Stacks model last-in-first-out work; queues model first-in-first-out work and staged processing. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to stacks and queues.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Validate nested brackets with a stack and traverse states with a queue.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Stacks and Queues in one precise sentence and name one assumption.
2. Create a two-step example of stacks and queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 03 / LANGUAGE AND NOTATION

Stacks and Queues: Language and Notation

Explain and apply stacks and queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for stacks and queues should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for stacks and queues.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Stacks and Queues in one precise sentence and name one assumption.
2. Create a two-step example of stacks and queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 03 / WORKED EXAMPLE A

Stacks and Queues: Worked Example A

Explain and apply stacks and queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Validate nested brackets with a stack and traverse states with a queue. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns stacks and queues into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Stacks and Queues in one precise sentence and name one assumption.
2. Create a two-step example of stacks and queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 03 / WORKED EXAMPLE B

Stacks and Queues: Worked Example B

Explain and apply stacks and queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Validate nested brackets with a stack and traverse states with a queue. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns stacks and queues into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Stacks and Queues in one precise sentence and name one assumption.
2. Create a two-step example of stacks and queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 03 / IMPLEMENTATION LAB

Stacks and Queues: Implementation Lab

Explain and apply stacks and queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of stacks and queues into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Validate nested brackets with a stack and traverse states with a queue. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Stacks and Queues in one precise sentence and name one assumption.
2. Create a two-step example of stacks and queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 03 / FAILURE MODES

Stacks and Queues: Failure Modes

Explain and apply stacks and queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how stacks and queues can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to stacks and queues. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Stacks and Queues in one precise sentence and name one assumption.
2. Create a two-step example of stacks and queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 03 / GUIDED PRACTICE

Stacks and Queues: Guided Practice

Explain and apply stacks and queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for stacks and queues removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Validate nested brackets with a stack and traverse states with a queue.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Stacks and Queues in one precise sentence and name one assumption.
2. Create a two-step example of stacks and queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 03 / INDEPENDENT PRACTICE

Stacks and Queues: Independent Practice

Explain and apply stacks and queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new stacks and queues task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Validate nested brackets with a stack and traverse states with a queue. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Stacks and Queues in one precise sentence and name one assumption.
2. Create a two-step example of stacks and queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 03 / MASTERY CHECK

Stacks and Queues: Mastery Check

Explain and apply stacks and queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of stacks and queues means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain stacks model last-in-first-out work; queues model first-in-first-out work and staged processing. Then solve and verify this scenario: Validate nested brackets with a stack and traverse states with a queue.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Stacks and Queues in one precise sentence and name one assumption.
2. Create a two-step example of stacks and queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 04 / ORIENTATION AND QUESTIONS

Hash Tables and Sets: Orientation and Questions

Explain and apply hash tables and sets using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Hashing trades additional space for near-constant expected lookup when keys are stable and equality is defined correctly. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Hash, Tables, Sets are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should hash tables and sets be used, and what observable evidence would make that choice defensible? Count tokens and detect duplicates in one pass.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Hash Tables and Sets in one precise sentence and name one assumption.
2. Create a two-step example of hash tables and sets and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 04 / FIRST-PRINCIPLES MODEL

Hash Tables and Sets: First-Principles Model

Explain and apply hash tables and sets using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Hashing trades additional space for near-constant expected lookup when keys are stable and equality is defined correctly. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to hash tables and sets.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Count tokens and detect duplicates in one pass.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Hash Tables and Sets in one precise sentence and name one assumption.
2. Create a two-step example of hash tables and sets and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 04 / LANGUAGE AND NOTATION

Hash Tables and Sets: Language and Notation

Explain and apply hash tables and sets using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for hash tables and sets should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for hash tables and sets.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Hash Tables and Sets in one precise sentence and name one assumption.
2. Create a two-step example of hash tables and sets and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 04 / WORKED EXAMPLE A

Hash Tables and Sets: Worked Example A

Explain and apply hash tables and sets using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Count tokens and detect duplicates in one pass. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns hash tables and sets into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Hash Tables and Sets in one precise sentence and name one assumption.
2. Create a two-step example of hash tables and sets and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 04 / WORKED EXAMPLE B

Hash Tables and Sets: Worked Example B

Explain and apply hash tables and sets using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Count tokens and detect duplicates in one pass. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns hash tables and sets into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Hash Tables and Sets in one precise sentence and name one assumption.
2. Create a two-step example of hash tables and sets and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 04 / IMPLEMENTATION LAB

Hash Tables and Sets: Implementation Lab

Explain and apply hash tables and sets using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of hash tables and sets into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Count tokens and detect duplicates in one pass. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Hash Tables and Sets in one precise sentence and name one assumption.
2. Create a two-step example of hash tables and sets and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 04 / FAILURE MODES

Hash Tables and Sets: Failure Modes

Explain and apply hash tables and sets using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how hash tables and sets can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to hash tables and sets. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Hash Tables and Sets in one precise sentence and name one assumption.
2. Create a two-step example of hash tables and sets and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 04 / GUIDED PRACTICE

Hash Tables and Sets: Guided Practice

Explain and apply hash tables and sets using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for hash tables and sets removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Count tokens and detect duplicates in one pass.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Hash Tables and Sets in one precise sentence and name one assumption.
2. Create a two-step example of hash tables and sets and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 04 / INDEPENDENT PRACTICE

Hash Tables and Sets: Independent Practice

Explain and apply hash tables and sets using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new hash tables and sets task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Count tokens and detect duplicates in one pass. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Hash Tables and Sets in one precise sentence and name one assumption.
2. Create a two-step example of hash tables and sets and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 04 / MASTERY CHECK

Hash Tables and Sets: Mastery Check

Explain and apply hash tables and sets using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of hash tables and sets means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain hashing trades additional space for near-constant expected lookup when keys are stable and equality is defined correctly. Then solve and verify this scenario: Count tokens and detect duplicates in one pass.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Hash Tables and Sets in one precise sentence and name one assumption.
2. Create a two-step example of hash tables and sets and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 05 / ORIENTATION AND QUESTIONS

Linked Structures: Orientation and Questions

Explain and apply linked structures using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Linked nodes support local insertion and removal but sacrifice direct indexing and cache-friendly traversal. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Linked, Structures are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should linked structures be used, and what observable evidence would make that choice defensible? Reverse a singly linked list while preserving the remaining chain.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Linked Structures in one precise sentence and name one assumption.
2. Create a two-step example of linked structures and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 05 / FIRST-PRINCIPLES MODEL

Linked Structures: First-Principles Model

Explain and apply linked structures using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Linked nodes support local insertion and removal but sacrifice direct indexing and cache-friendly traversal. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to linked structures.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Reverse a singly linked list while preserving the remaining chain.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Linked Structures in one precise sentence and name one assumption.
2. Create a two-step example of linked structures and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 05 / LANGUAGE AND NOTATION

Linked Structures: Language and Notation

Explain and apply linked structures using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for linked structures should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for linked structures.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Linked Structures in one precise sentence and name one assumption.
2. Create a two-step example of linked structures and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 05 / WORKED EXAMPLE A

Linked Structures: Worked Example A

Explain and apply linked structures using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Reverse a singly linked list while preserving the remaining chain. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns linked structures into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Linked Structures in one precise sentence and name one assumption.
2. Create a two-step example of linked structures and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 05 / WORKED EXAMPLE B

Linked Structures: Worked Example B

Explain and apply linked structures using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Reverse a singly linked list while preserving the remaining chain. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns linked structures into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Linked Structures in one precise sentence and name one assumption.
2. Create a two-step example of linked structures and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 05 / IMPLEMENTATION LAB

Linked Structures: Implementation Lab

Explain and apply linked structures using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of linked structures into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Reverse a singly linked list while preserving the remaining chain. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Linked Structures in one precise sentence and name one assumption.
2. Create a two-step example of linked structures and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 05 / FAILURE MODES

Linked Structures: Failure Modes

Explain and apply linked structures using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how linked structures can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to linked structures. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Linked Structures in one precise sentence and name one assumption.
2. Create a two-step example of linked structures and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 05 / GUIDED PRACTICE

Linked Structures: Guided Practice

Explain and apply linked structures using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for linked structures removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Reverse a singly linked list while preserving the remaining chain.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Linked Structures in one precise sentence and name one assumption.
2. Create a two-step example of linked structures and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 05 / INDEPENDENT PRACTICE

Linked Structures: Independent Practice

Explain and apply linked structures using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new linked structures task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Reverse a singly linked list while preserving the remaining chain. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Linked Structures in one precise sentence and name one assumption.
2. Create a two-step example of linked structures and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 05 / MASTERY CHECK

Linked Structures: Mastery Check

Explain and apply linked structures using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of linked structures means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain linked nodes support local insertion and removal but sacrifice direct indexing and cache-friendly traversal. Then solve and verify this scenario: Reverse a singly linked list while preserving the remaining chain.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Linked Structures in one precise sentence and name one assumption.
2. Create a two-step example of linked structures and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 06 / ORIENTATION AND QUESTIONS

Trees and Traversals: Orientation and Questions

Explain and apply trees and traversals using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Trees represent hierarchy; depth-first and breadth-first traversals expose different ordering and memory tradeoffs. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Trees, Traversals are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should trees and traversals be used, and what observable evidence would make that choice defensible? Compute the height of a binary tree and list nodes by level.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Trees and Traversals in one precise sentence and name one assumption.
2. Create a two-step example of trees and traversals and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 06 / FIRST-PRINCIPLES MODEL

Trees and Traversals: First-Principles Model

Explain and apply trees and traversals using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Trees represent hierarchy; depth-first and breadth-first traversals expose different ordering and memory tradeoffs. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to trees and traversals.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Compute the height of a binary tree and list nodes by level.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Trees and Traversals in one precise sentence and name one assumption.
2. Create a two-step example of trees and traversals and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 06 / LANGUAGE AND NOTATION

Trees and Traversals: Language and Notation

Explain and apply trees and traversals using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for trees and traversals should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for trees and traversals.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Trees and Traversals in one precise sentence and name one assumption.
2. Create a two-step example of trees and traversals and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 06 / WORKED EXAMPLE A

Trees and Traversals: Worked Example A

Explain and apply trees and traversals using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Compute the height of a binary tree and list nodes by level. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns trees and traversals into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Trees and Traversals in one precise sentence and name one assumption.
2. Create a two-step example of trees and traversals and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 06 / WORKED EXAMPLE B

Trees and Traversals: Worked Example B

Explain and apply trees and traversals using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Compute the height of a binary tree and list nodes by level. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns trees and traversals into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Trees and Traversals in one precise sentence and name one assumption.
2. Create a two-step example of trees and traversals and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 06 / IMPLEMENTATION LAB

Trees and Traversals: Implementation Lab

Explain and apply trees and traversals using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of trees and traversals into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Compute the height of a binary tree and list nodes by level. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Trees and Traversals in one precise sentence and name one assumption.
2. Create a two-step example of trees and traversals and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 06 / FAILURE MODES

Trees and Traversals: Failure Modes

Explain and apply trees and traversals using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how trees and traversals can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to trees and traversals. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Trees and Traversals in one precise sentence and name one assumption.
2. Create a two-step example of trees and traversals and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 06 / GUIDED PRACTICE

Trees and Traversals: Guided Practice

Explain and apply trees and traversals using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for trees and traversals removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Compute the height of a binary tree and list nodes by level.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Trees and Traversals in one precise sentence and name one assumption.
2. Create a two-step example of trees and traversals and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 06 / INDEPENDENT PRACTICE

Trees and Traversals: Independent Practice

Explain and apply trees and traversals using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new trees and traversals task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Compute the height of a binary tree and list nodes by level. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Trees and Traversals in one precise sentence and name one assumption.
2. Create a two-step example of trees and traversals and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 06 / MASTERY CHECK

Trees and Traversals: Mastery Check

Explain and apply trees and traversals using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of trees and traversals means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain trees represent hierarchy; depth-first and breadth-first traversals expose different ordering and memory tradeoffs. Then solve and verify this scenario: Compute the height of a binary tree and list nodes by level.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Trees and Traversals in one precise sentence and name one assumption.
2. Create a two-step example of trees and traversals and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 07 / ORIENTATION AND QUESTIONS

Heaps and Priority Queues: Orientation and Questions

Explain and apply heaps and priority queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

A heap maintains access to an extreme element without fully sorting all data. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Heaps, Priority, Queues are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should heaps and priority queues be used, and what observable evidence would make that choice defensible? Keep the five highest scores from a large stream.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Heaps and Priority Queues in one precise sentence and name one assumption.
2. Create a two-step example of heaps and priority queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 07 / FIRST-PRINCIPLES MODEL

Heaps and Priority Queues: First-Principles Model

Explain and apply heaps and priority queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

A heap maintains access to an extreme element without fully sorting all data. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to heaps and priority queues.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Keep the five highest scores from a large stream.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Heaps and Priority Queues in one precise sentence and name one assumption.
2. Create a two-step example of heaps and priority queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 07 / LANGUAGE AND NOTATION

Heaps and Priority Queues: Language and Notation

Explain and apply heaps and priority queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for heaps and priority queues should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for heaps and priority queues.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Heaps and Priority Queues in one precise sentence and name one assumption.
2. Create a two-step example of heaps and priority queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 07 / WORKED EXAMPLE A

Heaps and Priority Queues: Worked Example A

Explain and apply heaps and priority queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Keep the five highest scores from a large stream. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns heaps and priority queues into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Heaps and Priority Queues in one precise sentence and name one assumption.
2. Create a two-step example of heaps and priority queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 07 / WORKED EXAMPLE B

Heaps and Priority Queues: Worked Example B

Explain and apply heaps and priority queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Keep the five highest scores from a large stream. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns heaps and priority queues into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Heaps and Priority Queues in one precise sentence and name one assumption.
2. Create a two-step example of heaps and priority queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 07 / IMPLEMENTATION LAB

Heaps and Priority Queues: Implementation Lab

Explain and apply heaps and priority queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of heaps and priority queues into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Keep the five highest scores from a large stream. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Heaps and Priority Queues in one precise sentence and name one assumption.
2. Create a two-step example of heaps and priority queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 07 / FAILURE MODES

Heaps and Priority Queues: Failure Modes

Explain and apply heaps and priority queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how heaps and priority queues can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to heaps and priority queues. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Heaps and Priority Queues in one precise sentence and name one assumption.
2. Create a two-step example of heaps and priority queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 07 / GUIDED PRACTICE

Heaps and Priority Queues: Guided Practice

Explain and apply heaps and priority queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for heaps and priority queues removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Keep the five highest scores from a large stream.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Heaps and Priority Queues in one precise sentence and name one assumption.
2. Create a two-step example of heaps and priority queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 07 / INDEPENDENT PRACTICE

Heaps and Priority Queues: Independent Practice

Explain and apply heaps and priority queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new heaps and priority queues task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Keep the five highest scores from a large stream. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Heaps and Priority Queues in one precise sentence and name one assumption.
2. Create a two-step example of heaps and priority queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 07 / MASTERY CHECK

Heaps and Priority Queues: Mastery Check

Explain and apply heaps and priority queues using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of heaps and priority queues means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain a heap maintains access to an extreme element without fully sorting all data. Then solve and verify this scenario: Keep the five highest scores from a large stream.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Heaps and Priority Queues in one precise sentence and name one assumption.
2. Create a two-step example of heaps and priority queues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 08 / ORIENTATION AND QUESTIONS

Graphs and Networks: Orientation and Questions

Explain and apply graphs and networks using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Graphs model entities and relationships; representation determines the cost of traversal and neighbor queries. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Graphs, Networks are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should graphs and networks be used, and what observable evidence would make that choice defensible? Represent a small knowledge graph with adjacency lists.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Graphs and Networks in one precise sentence and name one assumption.
2. Create a two-step example of graphs and networks and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 08 / FIRST-PRINCIPLES MODEL

Graphs and Networks: First-Principles Model

Explain and apply graphs and networks using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Graphs model entities and relationships; representation determines the cost of traversal and neighbor queries. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to graphs and networks.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Represent a small knowledge graph with adjacency lists.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Graphs and Networks in one precise sentence and name one assumption.
2. Create a two-step example of graphs and networks and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 08 / LANGUAGE AND NOTATION

Graphs and Networks: Language and Notation

Explain and apply graphs and networks using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for graphs and networks should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for graphs and networks.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Graphs and Networks in one precise sentence and name one assumption.
2. Create a two-step example of graphs and networks and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 08 / WORKED EXAMPLE A

Graphs and Networks: Worked Example A

Explain and apply graphs and networks using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Represent a small knowledge graph with adjacency lists. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns graphs and networks into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Graphs and Networks in one precise sentence and name one assumption.
2. Create a two-step example of graphs and networks and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 08 / WORKED EXAMPLE B

Graphs and Networks: Worked Example B

Explain and apply graphs and networks using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Represent a small knowledge graph with adjacency lists. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns graphs and networks into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Graphs and Networks in one precise sentence and name one assumption.
2. Create a two-step example of graphs and networks and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 08 / IMPLEMENTATION LAB

Graphs and Networks: Implementation Lab

Explain and apply graphs and networks using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of graphs and networks into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Represent a small knowledge graph with adjacency lists. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Graphs and Networks in one precise sentence and name one assumption.
2. Create a two-step example of graphs and networks and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 08 / FAILURE MODES

Graphs and Networks: Failure Modes

Explain and apply graphs and networks using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how graphs and networks can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to graphs and networks. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Graphs and Networks in one precise sentence and name one assumption.
2. Create a two-step example of graphs and networks and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 08 / GUIDED PRACTICE

Graphs and Networks: Guided Practice

Explain and apply graphs and networks using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for graphs and networks removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Represent a small knowledge graph with adjacency lists.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Graphs and Networks in one precise sentence and name one assumption.
2. Create a two-step example of graphs and networks and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 08 / INDEPENDENT PRACTICE

Graphs and Networks: Independent Practice

Explain and apply graphs and networks using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new graphs and networks task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Represent a small knowledge graph with adjacency lists. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Graphs and Networks in one precise sentence and name one assumption.
2. Create a two-step example of graphs and networks and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 08 / MASTERY CHECK

Graphs and Networks: Mastery Check

Explain and apply graphs and networks using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of graphs and networks means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain graphs model entities and relationships; representation determines the cost of traversal and neighbor queries. Then solve and verify this scenario: Represent a small knowledge graph with adjacency lists.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Graphs and Networks in one precise sentence and name one assumption.
2. Create a two-step example of graphs and networks and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 09 / ORIENTATION AND QUESTIONS

Sorting and Selection: Orientation and Questions

Explain and apply sorting and selection using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Sorting creates order for later operations, while selection finds ranks without necessarily ordering everything. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Sorting, Selection are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should sorting and selection be used, and what observable evidence would make that choice defensible? Compare merge sort with quicksort on stability and worst-case behavior.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Sorting and Selection in one precise sentence and name one assumption.
2. Create a two-step example of sorting and selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 09 / FIRST-PRINCIPLES MODEL

Sorting and Selection: First-Principles Model

Explain and apply sorting and selection using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Sorting creates order for later operations, while selection finds ranks without necessarily ordering everything. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to sorting and selection.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Compare merge sort with quicksort on stability and worst-case behavior.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Sorting and Selection in one precise sentence and name one assumption.
2. Create a two-step example of sorting and selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 09 / LANGUAGE AND NOTATION

Sorting and Selection: Language and Notation

Explain and apply sorting and selection using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for sorting and selection should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for sorting and selection.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Sorting and Selection in one precise sentence and name one assumption.
2. Create a two-step example of sorting and selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 09 / WORKED EXAMPLE A

Sorting and Selection: Worked Example A

Explain and apply sorting and selection using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Compare merge sort with quicksort on stability and worst-case behavior. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns sorting and selection into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Sorting and Selection in one precise sentence and name one assumption.
2. Create a two-step example of sorting and selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 09 / WORKED EXAMPLE B

Sorting and Selection: Worked Example B

Explain and apply sorting and selection using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Compare merge sort with quicksort on stability and worst-case behavior. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns sorting and selection into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Sorting and Selection in one precise sentence and name one assumption.
2. Create a two-step example of sorting and selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 09 / IMPLEMENTATION LAB

Sorting and Selection: Implementation Lab

Explain and apply sorting and selection using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of sorting and selection into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Compare merge sort with quicksort on stability and worst-case behavior. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Sorting and Selection in one precise sentence and name one assumption.
2. Create a two-step example of sorting and selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 09 / FAILURE MODES

Sorting and Selection: Failure Modes

Explain and apply sorting and selection using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how sorting and selection can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to sorting and selection. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Sorting and Selection in one precise sentence and name one assumption.
2. Create a two-step example of sorting and selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 09 / GUIDED PRACTICE

Sorting and Selection: Guided Practice

Explain and apply sorting and selection using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for sorting and selection removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Compare merge sort with quicksort on stability and worst-case behavior.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Sorting and Selection in one precise sentence and name one assumption.
2. Create a two-step example of sorting and selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 09 / INDEPENDENT PRACTICE

Sorting and Selection: Independent Practice

Explain and apply sorting and selection using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new sorting and selection task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Compare merge sort with quicksort on stability and worst-case behavior. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Sorting and Selection in one precise sentence and name one assumption.
2. Create a two-step example of sorting and selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 09 / MASTERY CHECK

Sorting and Selection: Mastery Check

Explain and apply sorting and selection using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of sorting and selection means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain sorting creates order for later operations, while selection finds ranks without necessarily ordering everything. Then solve and verify this scenario: Compare merge sort with quicksort on stability and worst-case behavior.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Sorting and Selection in one precise sentence and name one assumption.
2. Create a two-step example of sorting and selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 10 / ORIENTATION AND QUESTIONS

Searching: Orientation and Questions

Explain and apply searching using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Search moves through a state or value space using ordering, heuristics, pruning, or stored structure. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Searching are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should searching be used, and what observable evidence would make that choice defensible? Use binary search for a boundary and breadth-first search for an unweighted path.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Searching in one precise sentence and name one assumption.
2. Create a two-step example of searching and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 10 / FIRST-PRINCIPLES MODEL

Searching: First-Principles Model

Explain and apply searching using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Search moves through a state or value space using ordering, heuristics, pruning, or stored structure. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to searching.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Use binary search for a boundary and breadth-first search for an unweighted path.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Searching in one precise sentence and name one assumption.
2. Create a two-step example of searching and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 10 / LANGUAGE AND NOTATION

Searching: Language and Notation

Explain and apply searching using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for searching should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for searching.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Searching in one precise sentence and name one assumption.
2. Create a two-step example of searching and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 10 / WORKED EXAMPLE A

Searching: Worked Example A

Explain and apply searching using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Use binary search for a boundary and breadth-first search for an unweighted path. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns searching into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Searching in one precise sentence and name one assumption.
2. Create a two-step example of searching and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 10 / WORKED EXAMPLE B

Searching: Worked Example B

Explain and apply searching using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Use binary search for a boundary and breadth-first search for an unweighted path. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns searching into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Searching in one precise sentence and name one assumption.
2. Create a two-step example of searching and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 10 / IMPLEMENTATION LAB

Searching: Implementation Lab

Explain and apply searching using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of searching into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Use binary search for a boundary and breadth-first search for an unweighted path. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Searching in one precise sentence and name one assumption.
2. Create a two-step example of searching and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 10 / FAILURE MODES

Searching: Failure Modes

Explain and apply searching using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how searching can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to searching. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Searching in one precise sentence and name one assumption.
2. Create a two-step example of searching and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 10 / GUIDED PRACTICE

Searching: Guided Practice

Explain and apply searching using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for searching removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Use binary search for a boundary and breadth-first search for an unweighted path.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Searching in one precise sentence and name one assumption.
2. Create a two-step example of searching and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 10 / INDEPENDENT PRACTICE

Searching: Independent Practice

Explain and apply searching using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new searching task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Use binary search for a boundary and breadth-first search for an unweighted path. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Searching in one precise sentence and name one assumption.
2. Create a two-step example of searching and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 10 / MASTERY CHECK

Searching: Mastery Check

Explain and apply searching using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of searching means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain search moves through a state or value space using ordering, heuristics, pruning, or stored structure. Then solve and verify this scenario: Use binary search for a boundary and breadth-first search for an unweighted path.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Searching in one precise sentence and name one assumption.
2. Create a two-step example of searching and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 11 / ORIENTATION AND QUESTIONS

Recursion and Divide-and-Conquer: Orientation and Questions

Explain and apply recursion and divide-and-conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Recursion solves a problem through smaller instances, requiring a base case and a decreasing measure. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Recursion, Divide, Conquer are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should recursion and divide-and-conquer be used, and what observable evidence would make that choice defensible? Trace merge sort and identify the recursion tree.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Recursion and Divide-and-Conquer in one precise sentence and name one assumption.
2. Create a two-step example of recursion and divide-and-conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 11 / FIRST-PRINCIPLES MODEL

Recursion and Divide-and-Conquer: First-Principles Model

Explain and apply recursion and divide-and-conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Recursion solves a problem through smaller instances, requiring a base case and a decreasing measure. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to recursion and divide-and-conquer.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Trace merge sort and identify the recursion tree.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Recursion and Divide-and-Conquer in one precise sentence and name one assumption.
2. Create a two-step example of recursion and divide-and-conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 11 / LANGUAGE AND NOTATION

Recursion and Divide-and-Conquer: Language and Notation

Explain and apply recursion and divide-and-conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for recursion and divide-and-conquer should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for recursion and divide-and-conquer.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Recursion and Divide-and-Conquer in one precise sentence and name one assumption.
2. Create a two-step example of recursion and divide-and-conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 11 / WORKED EXAMPLE A

Recursion and Divide-and-Conquer: Worked Example A

Explain and apply recursion and divide-and-conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Trace merge sort and identify the recursion tree. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns recursion and divide-and-conquer into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Recursion and Divide-and-Conquer in one precise sentence and name one assumption.
2. Create a two-step example of recursion and divide-and-conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 11 / WORKED EXAMPLE B

Recursion and Divide-and-Conquer: Worked Example B

Explain and apply recursion and divide-and-conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Trace merge sort and identify the recursion tree. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns recursion and divide-and-conquer into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Recursion and Divide-and-Conquer in one precise sentence and name one assumption.
2. Create a two-step example of recursion and divide-and-conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 11 / IMPLEMENTATION LAB

Recursion and Divide-and-Conquer: Implementation Lab

Explain and apply recursion and divide-and-conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of recursion and divide-and-conquer into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Trace merge sort and identify the recursion tree. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Recursion and Divide-and-Conquer in one precise sentence and name one assumption.
2. Create a two-step example of recursion and divide-and-conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 11 / FAILURE MODES

Recursion and Divide-and-Conquer: Failure Modes

Explain and apply recursion and divide-and-conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how recursion and divide-and-conquer can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to recursion and divide-and-conquer. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Recursion and Divide-and-Conquer in one precise sentence and name one assumption.
2. Create a two-step example of recursion and divide-and-conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 11 / GUIDED PRACTICE

Recursion and Divide-and-Conquer: Guided Practice

Explain and apply recursion and divide-and-conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for recursion and divide-and-conquer removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Trace merge sort and identify the recursion tree.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Recursion and Divide-and-Conquer in one precise sentence and name one assumption.
2. Create a two-step example of recursion and divide-and-conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 11 / INDEPENDENT PRACTICE

Recursion and Divide-and-Conquer: Independent Practice

Explain and apply recursion and divide-and-conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new recursion and divide-and-conquer task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Trace merge sort and identify the recursion tree. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Recursion and Divide-and-Conquer in one precise sentence and name one assumption.
2. Create a two-step example of recursion and divide-and-conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 11 / MASTERY CHECK

Recursion and Divide-and-Conquer: Mastery Check

Explain and apply recursion and divide-and-conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of recursion and divide-and-conquer means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain recursion solves a problem through smaller instances, requiring a base case and a decreasing measure. Then solve and verify this scenario: Trace merge sort and identify the recursion tree.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Recursion and Divide-and-Conquer in one precise sentence and name one assumption.
2. Create a two-step example of recursion and divide-and-conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 12 / ORIENTATION AND QUESTIONS

Greedy Algorithms: Orientation and Questions

Explain and apply greedy algorithms using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

A greedy method commits to a locally attractive choice and needs a proof that this preserves global optimality. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Greedy, Algorithms are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should greedy algorithms be used, and what observable evidence would make that choice defensible? Select the maximum number of non-overlapping intervals.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Greedy Algorithms in one precise sentence and name one assumption.
2. Create a two-step example of greedy algorithms and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 12 / FIRST-PRINCIPLES MODEL

Greedy Algorithms: First-Principles Model

Explain and apply greedy algorithms using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

A greedy method commits to a locally attractive choice and needs a proof that this preserves global optimality. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to greedy algorithms.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Select the maximum number of non-overlapping intervals.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Greedy Algorithms in one precise sentence and name one assumption.
2. Create a two-step example of greedy algorithms and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 12 / LANGUAGE AND NOTATION

Greedy Algorithms: Language and Notation

Explain and apply greedy algorithms using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for greedy algorithms should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for greedy algorithms.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Greedy Algorithms in one precise sentence and name one assumption.
2. Create a two-step example of greedy algorithms and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 12 / WORKED EXAMPLE A

Greedy Algorithms: Worked Example A

Explain and apply greedy algorithms using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Select the maximum number of non-overlapping intervals. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns greedy algorithms into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Greedy Algorithms in one precise sentence and name one assumption.
2. Create a two-step example of greedy algorithms and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 12 / WORKED EXAMPLE B

Greedy Algorithms: Worked Example B

Explain and apply greedy algorithms using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Select the maximum number of non-overlapping intervals. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns greedy algorithms into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Greedy Algorithms in one precise sentence and name one assumption.
2. Create a two-step example of greedy algorithms and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 12 / IMPLEMENTATION LAB

Greedy Algorithms: Implementation Lab

Explain and apply greedy algorithms using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of greedy algorithms into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Select the maximum number of non-overlapping intervals. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Greedy Algorithms in one precise sentence and name one assumption.
2. Create a two-step example of greedy algorithms and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 12 / FAILURE MODES

Greedy Algorithms: Failure Modes

Explain and apply greedy algorithms using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how greedy algorithms can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to greedy algorithms. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Greedy Algorithms in one precise sentence and name one assumption.
2. Create a two-step example of greedy algorithms and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 12 / GUIDED PRACTICE

Greedy Algorithms: Guided Practice

Explain and apply greedy algorithms using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for greedy algorithms removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Select the maximum number of non-overlapping intervals.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Greedy Algorithms in one precise sentence and name one assumption.
2. Create a two-step example of greedy algorithms and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 12 / INDEPENDENT PRACTICE

Greedy Algorithms: Independent Practice

Explain and apply greedy algorithms using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new greedy algorithms task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Select the maximum number of non-overlapping intervals. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Greedy Algorithms in one precise sentence and name one assumption.
2. Create a two-step example of greedy algorithms and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 12 / MASTERY CHECK

Greedy Algorithms: Mastery Check

Explain and apply greedy algorithms using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of greedy algorithms means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain a greedy method commits to a locally attractive choice and needs a proof that this preserves global optimality. Then solve and verify this scenario: Select the maximum number of non-overlapping intervals.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Greedy Algorithms in one precise sentence and name one assumption.
2. Create a two-step example of greedy algorithms and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 13 / ORIENTATION AND QUESTIONS

Dynamic Programming: Orientation and Questions

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Dynamic programming stores overlapping subproblem results and builds an answer from a recurrence and state definition. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Dynamic, Programming are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should dynamic programming be used, and what observable evidence would make that choice defensible? Find a minimum-cost path through a small grid.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 13 / FIRST-PRINCIPLES MODEL

Dynamic Programming: First-Principles Model

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Dynamic programming stores overlapping subproblem results and builds an answer from a recurrence and state definition. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to dynamic programming.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Find a minimum-cost path through a small grid.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 13 / LANGUAGE AND NOTATION

Dynamic Programming: Language and Notation

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for dynamic programming should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for dynamic programming.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 13 / WORKED EXAMPLE A

Dynamic Programming: Worked Example A

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Find a minimum-cost path through a small grid. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns dynamic programming into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 13 / WORKED EXAMPLE B

Dynamic Programming: Worked Example B

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Find a minimum-cost path through a small grid. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns dynamic programming into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 13 / IMPLEMENTATION LAB

Dynamic Programming: Implementation Lab

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of dynamic programming into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Find a minimum-cost path through a small grid. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 13 / FAILURE MODES

Dynamic Programming: Failure Modes

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how dynamic programming can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to dynamic programming. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 13 / GUIDED PRACTICE

Dynamic Programming: Guided Practice

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for dynamic programming removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Find a minimum-cost path through a small grid.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 13 / INDEPENDENT PRACTICE

Dynamic Programming: Independent Practice

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new dynamic programming task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Find a minimum-cost path through a small grid. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 13 / MASTERY CHECK

Dynamic Programming: Mastery Check

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of dynamic programming means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain dynamic programming stores overlapping subproblem results and builds an answer from a recurrence and state definition. Then solve and verify this scenario: Find a minimum-cost path through a small grid.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 14 / ORIENTATION AND QUESTIONS

Backtracking, Constraints, and AI Pipelines: Orientation and Questions

Explain and apply backtracking, constraints, and ai pipelines using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Backtracking explores choices and reverses failed decisions; constraints and pruning determine feasibility. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithms, Data Structures, and AI Problem Solving, the terms Backtracking, Constraints, Pipelines are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

REASONING AND EVIDENCE

Central question: When should backtracking, constraints, and ai pipelines be used, and what observable evidence would make that choice defensible? Solve a compact scheduling problem and connect search stages to an AI pipeline.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Backtracking, Constraints, and AI Pipelines in one precise sentence and name one assumption.
2. Create a two-step example of backtracking, constraints, and ai pipelines and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 14 / FIRST-PRINCIPLES MODEL

Backtracking, Constraints, and AI Pipelines: First-Principles Model

Explain and apply backtracking, constraints, and ai pipelines using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Backtracking explores choices and reverses failed decisions; constraints and pruning determine feasibility. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to backtracking, constraints, and ai pipelines.

REASONING AND EVIDENCE

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Solve a compact scheduling problem and connect search stages to an AI pipeline.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Backtracking, Constraints, and AI Pipelines in one precise sentence and name one assumption.
2. Create a two-step example of backtracking, constraints, and ai pipelines and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 14 / LANGUAGE AND NOTATION

Backtracking, Constraints, and AI Pipelines: Language and Notation

Explain and apply backtracking, constraints, and ai pipelines using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Technical language for backtracking, constraints, and ai pipelines should compress meaning, not hide it. Define every symbol, variable, shape, type, or state before using it. Distinguish a definition from an assumption and a measured result from an interpretation. When code or notation is introduced, read it in both directions: describe what each line means in plain English, then reconstruct the formal expression from the explanation. This habit reveals mismatched units, ambiguous names, and hidden assumptions early.

REASONING AND EVIDENCE

Notation audit: list the inputs, their types or dimensions, the transformation, the output, and one invariant for backtracking, constraints, and ai pipelines.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Backtracking, Constraints, and AI Pipelines in one precise sentence and name one assumption.
2. Create a two-step example of backtracking, constraints, and ai pipelines and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 14 / WORKED EXAMPLE A

Backtracking, Constraints, and AI Pipelines: Worked Example A

Explain and apply backtracking, constraints, and ai pipelines using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Solve a compact scheduling problem and connect search stages to an AI pipeline. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns backtracking, constraints, and ai pipelines into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Backtracking, Constraints, and AI Pipelines in one precise sentence and name one assumption.
2. Create a two-step example of backtracking, constraints, and ai pipelines and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 14 / WORKED EXAMPLE B

Backtracking, Constraints, and AI Pipelines: Worked Example B

Explain and apply backtracking, constraints, and ai pipelines using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Consider this task: Solve a compact scheduling problem and connect search stages to an AI pipeline. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns backtracking, constraints, and ai pipelines into a repeatable method rather than a memorized answer.

REASONING AND EVIDENCE

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Backtracking, Constraints, and AI Pipelines in one precise sentence and name one assumption.
2. Create a two-step example of backtracking, constraints, and ai pipelines and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 14 / IMPLEMENTATION LAB

Backtracking, Constraints, and AI Pipelines: Implementation Lab

Explain and apply backtracking, constraints, and ai pipelines using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Implementation converts the idea of backtracking, constraints, and ai pipelines into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

REASONING AND EVIDENCE

Lab brief: Solve a compact scheduling problem and connect search stages to an AI pipeline. Save the input, expected output, observed output, and a short explanation of any difference.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Backtracking, Constraints, and AI Pipelines in one precise sentence and name one assumption.
2. Create a two-step example of backtracking, constraints, and ai pipelines and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 14 / FAILURE MODES

Backtracking, Constraints, and AI Pipelines: Failure Modes

Explain and apply backtracking, constraints, and ai pipelines using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Failure analysis asks how backtracking, constraints, and ai pipelines can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

REASONING AND EVIDENCE

Failure probe: construct the smallest input that breaks the naive approach to backtracking, constraints, and ai pipelines. State the expected behavior and why the naive result is misleading.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Backtracking, Constraints, and AI Pipelines in one precise sentence and name one assumption.
2. Create a two-step example of backtracking, constraints, and ai pipelines and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 14 / GUIDED PRACTICE

Backtracking, Constraints, and AI Pipelines: Guided Practice

Explain and apply backtracking, constraints, and ai pipelines using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Guided practice for backtracking, constraints, and ai pipelines removes support gradually. First complete a labeled example. Then repeat the method with one label removed. Next solve a structurally similar problem with different values. At each stage, write the reason for the next operation before performing it. If the result is wrong, classify the error as conceptual, representational, procedural, or verification-related. This makes feedback specific enough to change the next attempt.

REASONING AND EVIDENCE

Practice sequence: reproduce the core method, vary one condition, predict the change, run the method, and compare prediction with evidence. Context: Solve a compact scheduling problem and connect search stages to an AI pipeline.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Backtracking, Constraints, and AI Pipelines in one precise sentence and name one assumption.
2. Create a two-step example of backtracking, constraints, and ai pipelines and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 14 / INDEPENDENT PRACTICE

Backtracking, Constraints, and AI Pipelines: Independent Practice

Explain and apply backtracking, constraints, and ai pipelines using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Independent practice demonstrates transfer. Solve a new backtracking, constraints, and ai pipelines task without copying the worked example. Before starting, write a short plan and define the evidence you will use to check the final result. After finishing, compare the solution against a simpler baseline and identify one limitation. A correct result without a verification path earns less confidence than a result that can be reproduced and challenged.

REASONING AND EVIDENCE

Independent challenge: adapt this setting to a larger or noisier case - Solve a compact scheduling problem and connect search stages to an AI pipeline. Explain the tradeoff introduced by the change.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Backtracking, Constraints, and AI Pipelines in one precise sentence and name one assumption.
2. Create a two-step example of backtracking, constraints, and ai pipelines and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

CHAPTER 14 / MASTERY CHECK

Backtracking, Constraints, and AI Pipelines: Mastery Check

Explain and apply backtracking, constraints, and ai pipelines using explicit inputs, assumptions, steps, evidence, and failure checks.

CORE EXPLANATION

Mastery of backtracking, constraints, and ai pipelines means more than recognizing its definition. You should be able to explain the mechanism, select it for an appropriate problem, implement or calculate a small example, diagnose a failure, and compare it with an alternative. Use the questions below without notes first, then review only the concept linked to an incorrect answer. Record both accuracy and the amount of help required.

REASONING AND EVIDENCE

Mastery evidence: explain backtracking explores choices and reverses failed decisions; constraints and pruning determine feasibility. Then solve and verify this scenario: Solve a compact scheduling problem and connect search stages to an AI pipeline.

IMPLEMENTATION SKETCH

```
procedure BFS(start):
    queue <- [start]
    seen <- {start}
    while queue is not empty:
        node <- remove_front(queue)
        for each neighbor of node:
            if neighbor not in seen:
                add neighbor to seen and queue
```

PRACTICE AND RETRIEVAL

1. Define Backtracking, Constraints, and AI Pipelines in one precise sentence and name one assumption.
2. Create a two-step example of backtracking, constraints, and ai pipelines and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

FRONT MATTER

Capstone Brief

EDITORIAL GUIDANCE

Combine the book's major skills into one local project. Define the problem, baseline, tests, evidence, limitations, and a reproducible run procedure.

READER NOTE

Use this book as a sequence, annotate key arguments, reproduce the examples, and keep a separate notebook for attempts, corrections, and reflections.

FRONT MATTER

Capstone Rubric

EDITORIAL GUIDANCE

Evaluate correctness, clarity, robustness, reproducibility, efficiency, accessibility, and honesty of limitations. Each criterion needs observable evidence.

READER NOTE

Use this book as a sequence, annotate key arguments, reproduce the examples, and keep a separate notebook for attempts, corrections, and reflections.

FRONT MATTER

Cumulative Review

EDITORIAL GUIDANCE

Revisit one mastery check from every chapter. Group mistakes by misconception and schedule a delayed second attempt.

READER NOTE

Use this book as a sequence, annotate key arguments, reproduce the examples, and keep a separate notebook for attempts, corrections, and reflections.

FRONT MATTER

Glossary Strategy

EDITORIAL GUIDANCE

Build a personal glossary containing definitions, a minimal example, a non-example, and a connection for every term that still feels vague.

READER NOTE

Use this book as a sequence, annotate key arguments, reproduce the examples, and keep a separate notebook for attempts, corrections, and reflections.

FRONT MATTER

Reference and Provenance Note

EDITORIAL GUIDANCE

This is original curriculum text created for Nous AI Academy. Verify technical examples during editorial review against official Python, NumPy, scikit-learn, PyTorch, and relevant standards documentation.

READER NOTE

Use this book as a sequence, annotate key arguments, reproduce the examples, and keep a separate notebook for attempts, corrections, and reflections.

FRONT MATTER

Next Steps

EDITORIAL GUIDANCE

Export your project, notes, mastery record, and mistake notebook. Continue to the next core book or the related optional deep-dive resources.

READER NOTE

Use this book as a sequence, annotate key arguments, reproduce the examples, and keep a separate notebook for attempts, corrections, and reflections.